📝 NextGen Campus — Complete Implementation Report

> ***Project:*** *NextGen Campus Management System*
> ***Date:*** *March 26, 2026*
> ***Objective:*** *Deploy Frontend on Vercel + Backend on Render*
> ***Repository:*** *github.com/pratikayalapujith/Nextgen_Campus_*

Table of Contents

1. Initial Project Analysis
2. Adding Production Dependencies
3. Making Backend Config Production-Ready
4. Making Frontend API URL Dynamic
5. Creating Render Blueprint File
6. Creating Vercel SPA Config
7. First Git Push & Render Build Failure
8. Fixing Render Start Command Error
9. Fixing Python 3.14 Compatibility Error
10. Auto-Seeding Database Without Shell Access
11. Fixing SQLite Fallback for Free Tier
12. Vercel Frontend Configuration

Step 1: Initial Project Analysis

What We Did

Examined the entire project structure to understand how the frontend and backend are organized, what technologies are used, and what needs to change for deployment.

Findings

Project Structure:

```
next campus/
├── backend/          ← Flask (Python) backend
│   ├── run.py        ← Entry point
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       ├── config.py
│       ├── extensions.py
│       ├── models/
│       ├── routes/
│       ├── seeds/
│       ├── services/
│       └── utils/
└── frontend/         ← React + Vite + TypeScript frontend
    ├── package.json
```

```
    ├── vite.config.ts
    └── src/
        ├── api/axios.ts
        ├── utils/constants.ts
        ├── context/AuthContext.tsx
        └── pages/
```

Key Observations:

1. **Backend** uses Flask with SQLAlchemy (SQLite locally), JWT authentication, and CORS
2. **Frontend** uses Vite + React + TypeScript + TailwindCSS + ShadCN UI
3. **API URL was hardcoded** to `http://127.0.0.1:5001/api` in the frontend — this must be made dynamic
4. **No production server** — Flask's built-in dev server was used (`python run.py`) — this is not suitable for production
5. **No PostgreSQL driver** — only SQLite was configured
6. **Config only had DevelopmentConfig** as the default — no auto-detection of production environment

Original Files Before Any Changes

`backend/requirements.txt` :

```
Flask==3.1.0
Flask-SQLAlchemy==3.1.1
Flask-Migrate==4.0.7
Flask-JWT-Extended==4.7.1
Flask-CORS==5.0.1
python-dotenv==1.0.1
Werkzeug==3.1.3
```

`backend/app/config.py` :

```python
import os
from datetime import timedelta

class Config:
    SECRET_KEY = os.getenv('SECRET_KEY', 'nextgen-campus-dev-secret-key')
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY', 'nextgen-jwt-dev-secret')
    JWT_ACCESS_TOKEN_EXPIRES = timedelta(minutes=30)
    JWT_REFRESH_TOKEN_EXPIRES = timedelta(days=30)

class DevelopmentConfig(Config):
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = 'sqlite:///campus.db'

class ProductionConfig(Config):
    DEBUG = False
    SQLALCHEMY_DATABASE_URI = os.getenv('DATABASE_URL')

class TestingConfig(Config):
```

```
    TESTING = True
    SQLALCHEMY_DATABASE_URI = 'sqlite:///:memory:'
```

`backend/app/__init__.py`:

```
from flask import Flask
from app.config import DevelopmentConfig
from app.extensions import db, migrate, jwt, cors

def create_app(config_class=DevelopmentConfig):
    app = Flask(__name__)
    app.config.from_object(config_class)

    db.init_app(app)
    migrate.init_app(app, db)
    jwt.init_app(app)
    cors.init_app(app, resources={r"/api/*": {"origins": "*"}})

    from app.models.token_blocklist import TokenBlocklist
    @jwt.token_in_blocklist_loader
    def check_if_token_revoked(jwt_header, jwt_payload):
        jti = jwt_payload["jti"]
        return TokenBlocklist.query.filter_by(jti=jti).first() is not None

    from app.routes import register_blueprints
    register_blueprints(app)

    return app
```

`frontend/src/utils/constants.ts`:

```
export const API_BASE = 'http://127.0.0.1:5001/api';
```

Step 2: Adding Production Dependencies

What We Did

Added `gunicorn` and `psycopg2-binary` to `backend/requirements.txt`.

Why

- **gunicorn** — A production-grade WSGI HTTP server. Flask's built-in server (`flask run` or `python run.py`) is meant for development only. It's single-threaded, not secure, and not performant. Gunicorn is what Render uses to serve Python web apps.
- **psycopg2-binary** — PostgreSQL database adapter for Python. Required because Render provides PostgreSQL databases, and SQLAlchemy needs a driver to communicate with PostgreSQL.

Change Made

File: `backend/requirements.txt`

```
 Flask==3.1.0
 Flask-SQLAlchemy==3.1.1
 Flask-Migrate==4.0.7
 Flask-JWT-Extended==4.7.1
 Flask-CORS==5.0.1
 python-dotenv==1.0.1
 Werkzeug==3.1.3
+gunicorn==22.0.0
+psycopg2-binary==2.9.9
```

Step 3: Making Backend Config Production-Ready

What We Did

Modified `backend/app/config.py` to handle Render's PostgreSQL URL format.

Why

Render provides database URLs in the format `postgres://user:pass@host/db`, but SQLAlchemy 1.4+ requires `postgresql://` (with the full word). Without this conversion, the app crashes with:

```
sqlalchemy.exc.NoSuchModuleError: Can't load plugin: sqlalchemy.dialects:postgres
```

Change Made

File: `backend/app/config.py`

```
 class ProductionConfig(Config):
     DEBUG = False
-    SQLALCHEMY_DATABASE_URI = os.getenv('DATABASE_URL')
+    db_url = os.getenv('DATABASE_URL')
+    if db_url and db_url.startswith('postgres://'):
+        db_url = db_url.replace('postgres://', 'postgresql://', 1)
+    SQLALCHEMY_DATABASE_URI = db_url
```

How This Works

1. Reads `DATABASE_URL` from environment variables
2. Checks if it starts with `postgres://` (Render's format)
3. Replaces only the first occurrence of `postgres://` with `postgresql://`
4. Sets the corrected URL as the database connection string

Step 4: Making Frontend API URL Dynamic

What We Did

Changed the hardcoded API URL in the frontend to read from Vite environment variables.

Why

The frontend was hardcoded to `http://127.0.0.1:5001/api`, which only works when running locally. On Vercel, the frontend needs to call the Render backend at its public URL (e.g., `https://nextgen-campus-1.onrender.com/api`).

Change Made

File: `frontend/src/utils/constants.ts`

```
-export const API_BASE = 'http://127.0.0.1:5001/api';
+export const API_BASE = import.meta.env.VITE_API_URL ||
'http://127.0.0.1:5001/api';
```

How This Works

1. `import.meta.env.VITE_API_URL` — Vite's way of reading environment variables. Only variables prefixed with `VITE_` are exposed to the frontend code.
2. At **build time**, Vite replaces `import.meta.env.VITE_API_URL` with the actual value of the environment variable.
3. The `||` (OR) operator provides a fallback: if `VITE_API_URL` is not set (local development), it defaults to `http://127.0.0.1:5001/api`.

How the Frontend Uses This

`frontend/src/api/axios.ts` creates an Axios instance with this base URL:

```
import { API_BASE } from '../utils/constants';

const api = axios.create({
  baseURL: API_BASE,  // ← All API calls go through this
  headers: { 'Content-Type': 'application/json' },
});
```

Every API call in the app (login, fetch students, etc.) uses this `api` instance, so changing `API_BASE` changes where ALL requests go.

Step 5: Creating Render Blueprint File

What We Did

Created `render.yaml` in the project root directory.

Why

This is Render's "Infrastructure as Code" file. If you use Render's **New → Blueprint** option, it reads this file and auto-creates the web service, database, and environment variables in one click.

File Created

File: `render.yaml` (root)

```
services:
  - type: web
```

```yaml
    name: nextgen-campus-backend
    env: python
    buildCommand: cd backend && pip install -r requirements.txt
    startCommand: cd backend && gunicorn run:app
    envVars:
      - key: FLASK_ENV
        value: production
      - key: SECRET_KEY
        generateValue: true
      - key: JWT_SECRET_KEY
        generateValue: true
      - key: DATABASE_URL
        fromDatabase:
          name: nextgen-campus-db
          property: connectionString

databases:
  - name: nextgen-campus-db
    databaseName: campus
    user: campus_user
```

Line-by-Line Explanation

Line	Meaning
`type: web`	This is a web service (not a worker, cron, etc.)
`env: python`	Use Python runtime
`buildCommand`	Runs during deployment to install packages
`startCommand`	Runs to start the application
`FLASK_ENV: production`	Tells Flask to use ProductionConfig
`generateValue: true`	Render auto-generates a random secret
`fromDatabase`	Auto-links the PostgreSQL connection string
`databases`	Creates a free PostgreSQL database

> ***Note:*** *This file was created but ultimately NOT used because the user created the Render service manually through the dashboard. When creating manually, you must set all these values in the UI instead.*

Step 6: Creating Vercel SPA Config

What We Did

Created `frontend/vercel.json` for Single Page Application routing.

Why

Without this file, navigating to any route like `/dashboard` or `/students` directly in the browser returns a **404 error** from Vercel. This is because:

1. Vercel looks for a file at that path (e.g., `dashboard/index.html`)
2. No such file exists — React Router handles routing on the client-side
3. Vercel returns 404

The rewrite rule tells Vercel: "For ANY URL, serve `index.html` and let the React app handle the routing."

File Created

File: `frontend/vercel.json`

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

How It Works

- `"source": "/(.*)"` — Matches any URL path (regex: any character, any number of times)
- `"destination": "/index.html"` — Serves the main HTML file for all requests
- React Router inside `index.html` then reads the URL and renders the correct component

---

Step 7: First Git Push & Render Build Failure

What We Did

Committed all changes and pushed to GitHub.

Git Commands Run

```
git add .
git commit -m "feat: setup Vercel and Render deployment configs"
git push --set-upstream origin main
```

Commit Details

- **Commit hash:** `7fa3b93`
- **Files changed:** 5
  - `backend/requirements.txt` (modified)
  - `backend/app/config.py` (modified)
  - `frontend/src/utils/constants.ts` (modified)
  - `render.yaml` (new)
  - `frontend/vercel.json` (new)

❌ Error Encountered on Render

```
ERROR: Could not open requirements file: [Errno 2]
No such file or directory: 'requirements.txt'
==> Build failed 😞
```

Root Cause

Render was looking for `requirements.txt` in the **repository root**, but it's inside the `backend/` folder. The build command was `pip install -r requirements.txt` without navigating to `backend/` first.

Fix Applied

Updated `render.yaml` to explicitly navigate into the `backend/` directory:

```diff
-    rootDir: backend
-    buildCommand: pip install -r requirements.txt
-    startCommand: gunicorn run:app
+    buildCommand: cd backend && pip install -r requirements.txt
+    startCommand: cd backend && gunicorn run:app
```

Git Push

```
git commit -m "fix: modify render command paths to navigate to backend explicitly"
git push
```

- **Commit hash:** `92a3543`

Dashboard Manual Fix

Since the user created the service manually (not via Blueprint), the `render.yaml` changes were NOT automatically applied. The user had to manually update the Build Command and Start Command in the Render Dashboard settings.

Step 8: Fixing Render Start Command Error

❌ Error Encountered

```
ModuleNotFoundError: No module named 'run'
```

Root Cause

Even though the Build Command used `cd backend && pip install -r requirements.txt`, the Start Command `cd backend && gunicorn run:app` didn't work properly because Gunicorn's module import happens from the working directory where Gunicorn was initially invoked (the repo root), not from where `cd` navigated to.

Fix Applied

Changed the Start Command in the Render Dashboard to use Gunicorn's built-in `--chdir` flag:

```
gunicorn --chdir backend run:app
```

How `--chdir` Works

- `--chdir backend` — Tells Gunicorn to change its working directory to `backend/` before importing modules
- `run:app` — Import the `app` variable from `run.py`
- This is different from `cd backend &&` because `--chdir` affects Gunicorn's internal module resolution, not just the shell directory

---

Step 9: Fixing Python 3.14 Compatibility Error

❌ Error Encountered

```
ImportError: /opt/render/project/src/.venv/lib/python3.14/site-packages/
psycopg2/_psycopg.cpython-314-x86_64-linux-gnu.so:
undefined symbol: _PyInterpreterState_Get
```

Root Cause

Render defaulted to **Python 3.14** (the very latest version). The `psycopg2-binary` package has compiled C extensions (`.so` files) that are not yet compatible with Python 3.14's internal API changes. The symbol `_PyInterpreterState_Get` was removed or renamed in Python 3.14.

Fix Applied

Created a `.python-version` file in the repository root to pin Python to a stable version:

File: `.python-version` (root)

```
3.11.11
```

How It Works

Render automatically detects `.python-version` in the repository root and installs that specific Python version instead of the default.

Why 3.11?

- Python 3.11 is a stable, well-tested LTS release
- All packages in `requirements.txt` are fully compatible with it
- `psycopg2-binary` has pre-compiled wheels for Python 3.11

Git Push

```
git commit -m "fix: pin Python 3.11 for psycopg2 compatibility on Render"
git push
```

- **Commit hash:** `46b76d3`

Step 10: Auto-Seeding Database Without Shell Access

Problem

After deploying, the database was empty — no users, no departments, no seed data. The user needed to run `flask init-db` in a terminal on Render to create tables and seed data. But **Render's free plan does not include Shell access** (it's a paid feature).

What We Did

Modified `backend/app/__init__.py` to automatically create tables and seed the database on the very first startup.

Change Made

File: `backend/app/__init__.py`

```diff
+import os
 from flask import Flask
-from app.config import DevelopmentConfig
+from app.config import DevelopmentConfig, ProductionConfig
 from app.extensions import db, migrate, jwt, cors

-def create_app(config_class=DevelopmentConfig):
+def create_app(config_class=None):
+    if config_class is None:
+        if os.getenv('FLASK_ENV') == 'production' or os.getenv('RENDER'):
+            config_class = ProductionConfig
+        else:
+            config_class = DevelopmentConfig
+
     app = Flask(__name__)
     app.config.from_object(config_class)
     # ... (extensions and routes unchanged) ...

+    # Auto-initialize database on first startup
+    with app.app_context():
+        try:
+            from app.models.user import User
+            db.create_all()
+            if not User.query.first():
+                print('🔧 No users found — seeding database...')
+                from app.seeds.seed_data import seed
+                seed()
+            else:
+                print('✅ Database already initialized.')
+        except Exception as e:
+            print(f'⚠️ DB init skipped: {e}')
+
     return app
```

Logic Explained Step-by-Step

Part 1: Auto-detect environment

```
if config_class is None:
    if os.getenv('FLASK_ENV') == 'production' or os.getenv('RENDER'):
        config_class = ProductionConfig
    else:
        config_class = DevelopmentConfig
```

- Before this change, the app always used `DevelopmentConfig` (SQLite) unless manually overridden
- Now it checks for two environment variables:
  - `FLASK_ENV=production` — Set manually in Render env vars
  - `RENDER` — Automatically set by Render on all its services (always present)
- If either is found → use `ProductionConfig` (PostgreSQL)
- Otherwise → use `DevelopmentConfig` (SQLite, for local development)

Part 2: Auto-seed database

```
with app.app_context():
    try:
        from app.models.user import User
        db.create_all()
        if not User.query.first():
            seed()
    except Exception as e:
        print(f'⚠️ DB init skipped: {e}')
```

- `with app.app_context()` — Required by Flask-SQLAlchemy to access the database
- `db.create_all()` — Creates all tables defined in the models if they don't exist. If they already exist, does nothing.
- `User.query.first()` — Checks if ANY user exists in the database
- If no users → runs `seed()` which creates:
  - 1 Admin user
  - 5 Departments
  - 6 Faculty members
  - 12 Students
  - 7 Facilities
  - 11 Timetable entries
  - 5 Notices
- If users exist → skips seeding (prevents duplicates on restart)
- `try/except` — If the database connection fails, the app still starts (doesn't crash)

Git Push

```
git commit -m "feat: auto-initialize and seed database on startup (no Shell needed)"
git push
```

- **Commit hash:** `15e4108`

Result in Render Logs

```
🔧 No users found — seeding database...
✅ Test data seeded successfully!
==> Your service is live 🎉
```

On subsequent restarts:

```
✅ Database already initialized.
```

Step 11: Fixing SQLite Fallback for Free Tier

Problem

When `DATABASE_URL` was not set in the environment, `ProductionConfig` had `SQLALCHEMY_DATABASE_URI = None`, which caused the app to crash.

What We Did

Added a fallback to SQLite if no `DATABASE_URL` is configured.

Change Made

File: `backend/app/config.py`

```
 class ProductionConfig(Config):
     DEBUG = False
     db_url = os.getenv('DATABASE_URL')
     if db_url and db_url.startswith('postgres://'):
         db_url = db_url.replace('postgres://', 'postgresql://', 1)
-    SQLALCHEMY_DATABASE_URI = db_url
+    SQLALCHEMY_DATABASE_URI = db_url or 'sqlite:///campus.db'
```

How `or` Works Here

```
db_url or 'sqlite:///campus.db'
```

- If `db_url` is a non-empty string (PostgreSQL URL exists) → uses PostgreSQL
- If `db_url` is `None` or empty string → falls back to SQLite

Git Push

```
git commit -m "fix: fallback to SQLite when no DATABASE_URL is set on Render"
git push
```

- **Commit hash:** `1c3c856`

Step 12: Vercel Frontend Configuration

What We Did

Configured the Vercel project to correctly build and serve the frontend.

Vercel Dashboard Settings

Setting	Value	Why
Root Directory	`frontend`	The frontend code is in `frontend/`, not the repo root
Framework Preset	`Vite`	Tells Vercel to use Vite's build process
Build Command	`npm run build`	Runs `vite build` to create the production bundle
Output Directory	`dist`	Vite outputs built files to `dist/`

Environment Variable

Key	Value
`VITE_API_URL`	`https://nextgen-campus-1.onrender.com/api`

Why `/api` at the End

All backend routes are registered with the `/api` prefix:

```
auth_bp = Blueprint('auth', __name__, url_prefix='/api/auth')
students_bp = Blueprint('students', __name__, url_prefix='/api/students')
# etc.
```

The frontend Axios instance uses `API_BASE` as `baseURL`, and then appends route-specific paths:

```
api.post('/auth/login', { email, password })
// Full URL becomes: https://nextgen-campus-1.onrender.com/api/auth/login
```

❌ Initial Error: 404 NOT_FOUND on Vercel

Cause: Root Directory was not set to `frontend`, so Vercel tried to build from the repo root and found no `package.json` or `index.html`.
Fix: Set Root Directory to `frontend` in Vercel Settings → General.

Redeployment Requirement

Vite bakes `VITE_*` environment variables into the JavaScript bundle at **build time**. This means:

- Adding or changing `VITE_API_URL` requires a **full redeploy**
- Simply restarting the deployment won't update the value
- Go to Deployments → click ⋮ → Redeploy

📊 Complete Git History

#	Commit Hash	Message	Files Changed

1	`7fa3b93`	`feat: setup Vercel and Render deployment configs`	5 files
2	`92a3543`	`fix: modify render command paths to navigate to backend explicitly`	1 file
3	`15e4108`	`feat: auto-initialize and seed database on startup (no Shell needed)`	1 file
4	`46b76d3`	`fix: pin Python 3.11 for psycopg2 compatibility on Render`	1 file
5	`1c3c856`	`fix: fallback to SQLite when no DATABASE_URL is set on Render`	1 file

🔗 Final Architecture

```
┌──────────────────────────────────────────────────────────┐
│                        USER                              │
│                    (Web Browser)                         │
└──────────────────────────────────────────────────────────┘
                        │
                        │
                        │ HTTPS
                        ▼
┌──────────────────────────────────────────────────────────┐
│              VERCEL (Frontend)                           │
│                                                          │
│  URL: https://nextgen-campus.vercel.app                  │
│                                                          │
│   ┌──────────────────────────────────────────┐          │
│   │  Built Files (dist/)                     │          │
│   │  ├── index.html                          │          │
│   │  ├── assets/index-xxxxx.js  ← React app  │          │
│   │  └── assets/index-xxxxx.css ← Styles     │          │
│   └──────────────────────────────────────────┘          │
│                                                          │
│  vercel.json: Rewrites all routes → index.html           │
│  VITE_API_URL baked into JS at build time                │
└──────────────────────────────────────────────────────────┘
                        │
                        │
                        │ HTTPS (API calls via Axios)
                        │ Authorization: Bearer <JWT>
                        ▼
┌──────────────────────────────────────────────────────────┐
│              RENDER (Backend)                            │
│                                                          │
│  URL: https://nextgen-campus-1.onrender.com              │
│                                                          │
│   ┌──────────────────────────────────────────┐          │
│   │  Gunicorn (WSGI Server)                  │          │
│   │  └── run.py                              │          │
│   │      └── create_app()                    │          │
│   │          ├── Auto-detect: ProductionConfig  │       │
```

```
│  │          ├── Init: SQLAlchemy, JWT, CORS      │         │
│  │          ├── Register: 11 API Blueprints      │         │
│  │          └── Auto-seed: if no users exist     │         │
│  └──────────────────────────────────────────────┘         │
│                                                            │
│  Environment Variables:                                    │
│  ├── DATABASE_URL      → PostgreSQL connection             │
│  ├── SECRET_KEY        → Flask security                    │
│  ├── JWT_SECRET_KEY    → Token signing                     │
│  └── RENDER (auto-set) → Triggers ProductionConfig         │
└────────────────────────┬──────────────────────────────────┘
                         │
                         │ postgresql:// (Internal Network)
                         ▼
┌──────────────────────────────────────────────────────────┐
│           RENDER PostgreSQL Database                      │
│                                                           │
│  Tables:                                                  │
│  ├── users            (admin, faculty, student)           │
│  ├── departments      (CSE, ECE, ME, CE, EEE)             │
│  ├── faculty          (linked to users + departments)     │
│  ├── students         (linked to users + departments)     │
│  ├── subjects         (linked to departments + faculty)   │
│  ├── timetable_entries (day, time, room, subject)         │
│  ├── attendance       (student + subject + date)          │
│  ├── notices          (title, content, category)          │
│  ├── facilities       (labs, halls, sports)               │
│  ├── bookings         (facility + user + time)            │
│  └── token_blocklist  (revoked JWT tokens)                │
└──────────────────────────────────────────────────────────┘
```

🔐 Authentication Flow (Detailed)

Login Request

```
Frontend                          Backend
   │                                 │
   │  POST /api/auth/login           │
   │  {email, password}              │
   │──────────────────────────────►│
   │                                 │
   │                      ┌──────────┤
   │                      │ 1. Query user by email
   │                      │ 2. check_password(hash, input)
   │                      │ 3. Create JWT access_token (30 min)
   │                      │ 4. Create JWT refresh_token (30 days)
   │                      └──────────┤
   │                                 │
   │  {access_token,                 │
   │   refresh_token,                │
   │   user: {id, name, role}}       │
```

```
   │◄──────────────────────────────│
   │                               │
   │  Store tokens in localStorage  │
   │  Set user in AuthContext        │
   │  Redirect to /dashboard        │
```

Authenticated Request

```
Frontend                          Backend
   │                               │
   │  GET /api/students             │
   │  Authorization: Bearer <token> │
   │──────────────────────────────►│
   │                               │
   │                     ┌─────────┤
   │                     │ 1. Extract JWT from header
   │                     │ 2. Verify signature with JWT_SECRET_KEY
   │                     │ 3. Check token not in blocklist
   │                     │ 4. Check token not expired
   │                     │ 5. Query database
   │                     └─────────┤
   │                               │
   │  [{student1}, {student2}...]   │
   │◄──────────────────────────────│
```

Token Refresh (Automatic)

```
Frontend                          Backend
   │                               │
   │  GET /api/students             │
   │  Authorization: Bearer <expired_token>
   │──────────────────────────────►│
   │                               │
   │  401 Unauthorized              │
   │◄──────────────────────────────│
   │                               │
   │  (Axios interceptor kicks in)  │
   │                               │
   │  POST /api/auth/refresh        │
   │  Authorization: Bearer <refresh_token>
   │──────────────────────────────►│
   │                               │
   │  {access_token: <new_token>}   │
   │◄──────────────────────────────│
   │                               │
   │  (Retry original request)      │
   │  GET /api/students             │
   │  Authorization: Bearer <new_token>
   │──────────────────────────────►│
   │                               │
```

```
    |  [{student1}, {student2}...]   |
    |◄──────────────────────────────┤
```

🐛 All Errors Encountered & How They Were Fixed

Error 1: `requirements.txt` not found

```
ERROR: Could not open requirements file: [Errno 2]
No such file or directory: 'requirements.txt'
```

- **When:** First Render deployment
- **Root Cause:** Build command didn't navigate to `backend/` folder
- **Fix:** Changed build command to `cd backend && pip install -r requirements.txt`

Error 2: `No module named 'run'`

```
ModuleNotFoundError: No module named 'run'
```

- **When:** After build succeeded, start command failed
- **Root Cause:** `cd backend && gunicorn run:app` — shell `cd` doesn't affect Gunicorn's Python import system
- **Fix:** Changed start command to `gunicorn --chdir backend run:app`

Error 3: `psycopg2 undefined symbol`

```
ImportError: undefined symbol: _PyInterpreterState_Get
```

- **When:** After fixing start command, app crashed on import
- **Root Cause:** Python 3.14 incompatible with psycopg2-binary's compiled C extensions
- **Fix:** Created `.python-version` file with `3.11.11`

Error 4: `Invalid email or password`

```
Login attempt returns 401 on deployed site
```

- **When:** After successful deployment, trying to log in
- **Root Cause:** Database was empty — no seed data. Free Render plan has no Shell to run `flask init-db`.
- **Fix:** Added auto-seed logic to `create_app()` that runs on first startup

Error 5: Vercel `404 NOT_FOUND`

```
404: NOT_FOUND
Code: 'NOT_FOUND'
```

- **When:** Opening the Vercel deployment URL
- **Root Cause:** Vercel's Root Directory was not set to `frontend`, so it couldn't find the build files
- **Fix:** Set Root Directory to `frontend` in Vercel Settings

📦 Final State of All Modified/Created Files

Files Created (3)

1. `.python-version` — Pins Python 3.11.11
2. `render.yaml` — Render Blueprint config
3. `frontend/vercel.json` — SPA rewrite rules

Files Modified (3)

1. `backend/requirements.txt` — Added gunicorn + psycopg2-binary
2. `backend/app/config.py` — PostgreSQL URL conversion + SQLite fallback
3. `backend/app/__init__.py` — Auto environment detection + auto database seeding
4. `frontend/src/utils/constants.ts` — Dynamic API URL from env variable

🔗 Live URLs

Service	URL
Frontend	https://nextgen-campus.vercel.app
Backend	https://nextgen-campus-1.onrender.com
API Base	https://nextgen-campus-1.onrender.com/api

Demo Login Credentials

Role	Email	Password
Admin	`admin@campus.com`	`admin123`
Faculty	`rajesh.kumar@campus.com`	`faculty123`
Student	`rahul.verma@campus.com`	`student123`